The BBC Master 512

The Tube and the 80186 Co-processor

This article decribes the "low-level" workings of the Master 512 system. It covers how the parts which make up the co-processor relate to one another, and in particular gives some detail about the operation of the Tube on the 512, especially in relation to data transfers.

This is almost certainly the most technical of the articles in this collection. It will probably be difficult to follow without some understanding of the way that digital systems are put together. I must confess that I do not expect many people will want to make direct use of the information in this document, but there may be a number who would like to know a bit more about how their machine works. Much of the detail here I found out myself by trial and error, and by a close examination of the various pieces of information that are about. I do not see why anyone else should have to go through the same process (unless they really want to!)

As with all the other articles that make up this set, no attempt is made here to cover information that can be obtained elsewhere. However, some of this information is tucked away in rather obscure corners. If you want to get the best out of this article I would recommend that you read it along with a data sheet on the 80186 processor. I shall not cover the electronic details like tolerance levels and bus timings, but I will need to refer to the "internal peripherals" which are part of the design of the 80186. These are unique to this chip, and are not found in other processors of the 8086 family. I shall outline the relevant ones of these below, but it would take up too much space to describe them in detail. The data sheet is the only place I know of where you can find out about them.

Since I shall be describing the interconnections between the various parts of the system, you will probably also find it helpful to have a circuit diagram of the co-processor board. Information about the Tube in general, and from the host's side, will also be useful. This can be gleaned from Robin Burton's *Master 512 Technical Guide* and also from Chapter 12 of Watford's *Advanced Reference Manual for the BBC Master* and Chapter 18 of the *New Advanced User Guide*. Not everything in these books is accurate, though!

In all that follows the "BBC Micro part" of a Master 512 system will be called the "host". This is the 6502 processor along with its memory, software and interfaces. It can be either a Model B, B+ or Master 128. The "512 part" will be called the "parasite". This is the 80816 chip, together with its memory and software.

Parts of the System

The Master 512 co-processor system comprises three main parts. These are:

- The 80186 processor itself
- The memory – 512kb (or 1 Mb) of RAM and 16kb of ROM
- The Tube interface chip (the ULA)

In addition there are, of course, the connections between these (including various smaller chips). Naturally, a complete system also needs a BBC Micro host. There will not be much detail about the host in what follows, but it should not be forgotten! Furthermore, for the system to work at all there must be software. A Tube-based system relies on two processors communicating with one another, so each of them needs software and it must control them in a mutually compatible way.

The 80186 Processor

The 80186 is a complex chip. In addition to the processor unit, it contains several functions usually supplied by separate chips when other processors are in use. These extra functions operate like peripherals although they are integral to the main chip, and so they are known as "internal peripherals". These internal peripherals are the main way that the 80186 differs from all the other processors of the 8086 family. This includes both the older chips (the 8086 and 8088) and the newer ones (the 80286 and later chips).

The data sheet on the chip gives full details of all the internal peripherals, describing how to connect them up, response speeds, *etc*, as well as programmers' details about how to operate them. Study that if you want to know everything, but the sheet is quite difficult to follow without quite a lot of experience. For this reason I have described below the details which appear to be relevant in the Master 512.

The main constituents of the 80186 are:

- The processor unit itself
- Three timers
- Two Direct Memory Access (DMA) units
- A system for controlling and prioritizing hardware interrupts
- A set of programmable "chip select" lines

All of these apart from the processor unit count as internal peripherals, though the "chip selects" and, to some extent, the interrupt system are closely linked to the processor operation.

All the internal peripherals are controlled by writing to or reading from a control block. The control block is a block of 256 bytes, and it is normally the top page (page FFh) of I/O space. This means that you access it with IN and OUT instructions using DX and having DH=FFh. The control block can, in theory, be moved by writing a value to a "relocation register" (itself part of the control block), and it can be assigned to any page of I/O space or of memory. I can see no useful purpose in moving it on the 512.

The various parts of the 80186 are described next in a bit more detail:

The Processor Unit

This is very like the original 8086. It is a bit faster – it runs at a higher clock rate and the same instructions take fewer clock cycles. Also it provides a few extra instructions. The extra ones are:

`PUSH` an immediate value
`PUSHA`
`POPA`
`IMUL` by an immediate value
Shift/Rotate by an immediate value
`INS`
`OUTS`
`ENTER`
`LEAVE`
`BOUND`

The 80286 and later processors all include these, so references on them (which are easy to obtain) will give details of the extra instructions. The 80186 does not, though, provide any implementation of the "protected mode" operation of the 80286.

One other extra feature over the 8086 is that an unknown opcode causes an interrupt (Interrupt 6), and the `ESC` instruction can be made to cause an interrupt if desired.

The Timers

The 80186 contains three timers known (unsurprisingly) as Timer 0, Timer 1 and Timer 2. Timers 0 and 1 operate identically to one another. Timer 2 is rather more limited. People familiar with the BBC Micro host may well find the operation of the three timers rather similar to that of the timers in the 6522 VIAs. They each have an internal count register, which is incremented until it reaches a pre-set value, and at that point the action happens.

Timers 0 and 1 both have an input pin and an output pin. Each of them can be configured to run as a normal timer, taking its input from the system clock, or else to count pulses on its input pin. A third option is for the timer to take its input from Timer 2. When using the system clock, each timer increments its counter once every four clock cycles, so it runs at 2.5 MHz on the (10 MHz) Master 512. These timers can both be set to run continuously, and so provide a regular stream of pulses, or else to give a single pulse after a pre-set time. They each have two independent maximum count registers, and can be set up to use only one of them, or to use the two alternately. Each of them will deliver a pulse on its output pin when the maximum count is reached, and each can be set up to cause an interrupt at this point.

Timer 2 does not have any associated pins, and can only act as a normal timer. (It also runs at one quarter of the system clock rate.) It is also simpler than the other two in that it only has one maximum count register. Like the other two timers, it can be set up either to run continuously or to give a "single shot". Since it has no output pin, it can only be used to control functions internal to the 80186 itself. It can be used in three ways: either to generate interrupts, or to control the rate of one or both of the other two timers (see above), or else to control the rate of operation of the DMA units.

The detail of how to control the timers is not described here. See the data sheet if you want to do anything with them.

Connection of the Timers on the Master 512

On the 512 the input pins for Timers 0 and 1 are not connected. This means that these timers cannot be used to count pulses. They are normally set up to run as normal timers, using the system clock, and they do not generate interrupts (or not directly – see comment on Timer 1 in next paragraph).

Both of the output pins of these two timers are connected. The output of Timer 0 is used to provide a regular pulse on the Row Address Strobe and Column Address Strobe lines of the 512's RAM. This is needed to keep the RAM "alive" and I would not recommend playing around with it. The output of Timer 1 is connected to the NMI input line of the 80186. The reason for this is rather obscure, and it is outlined under the description of Interrupt 2 in the article about DOS-Plus interrupts.

Timer 2 is not normally used on the 512, and is available for program use if required.

The DMA Units

The Direct Memory Access (or DMA) units provide a means of transferring data quickly from one place to another. They can copy data from one area of memory to another, or else to or from an I/O port (or indeed directly from an input port to an output port). These units work independently of the processor unit and of one another, except for the limitations imposed by the address/data bus. The units can thus be set up to move data around while the processor is busy doing something else.

As might be expected, the two DMA units are known as DMA0 and DMA1. Each unit may be set up to transfer data continuously (*ie* as fast as possible, which will work out at 2.5Mb per second). Alternatively it may be set to transfer in response to a signal. This signal can be provided internally by Timer 2 (see above), or else externally. The external signal pins are known as DRQ0 and DRQ1, one for each unit. (DRQ stands for Data ReQuest.)

Again, see the data sheet for details of how to program these units.

The DMA Units on the Master 512

On the Master 512, DRQ0 is connected to an output line of the Tube chip, known as DRQ. Through this, DMA0 is used extensively for data transfers across the Tube. See below for more details. Because of this it would almost certainly be unwise to try to use DMA0 for anything else. DMA1 is not used by the system, though, and could be used in programs. DRQ1 is connected to the +5V line, so will issue continuous DMA requests. This means that DMA1 can only be used in unsynchronized (free-running) mode, or else controlled by Timer 2.

The Interrupt Controller

The hardware interrupt system of the 80186 can, in fact, be set up in four different ways. These are known as Normal Fully Nested Mode, Cascade Mode, Special Fully Nested Mode, and iRMX-86 Mode. On the Master 512 only Normal Fully Nested Mode is used, and no attempt is made here to describe any of the others.

Five input signal pins exist on the 80186 to allow external devices to generate interrupts. (Interrupts can also be generated internally by the Timers, the DMA units, and of course by `INT` instructions and program conditions like "division by zero".)

The signal pins are the NMI (Non-Maskable Interrupt) pin, and pins for four maskable interrupts known as INT0 to INT3. A pulse on the NMI pin (actually a low-to-high transition) causes the processor to execute Interrupt 2, and this cannot be prevented. Pulses on the other interrupt pins cause one of Interrupts Ch to Fh. (Interrupt Ch for INT0, Interrupt Dh for INT1, *etc*.) Any of these interrupts can be prevented by clearing the interrupt enable flag with a `CLI` instruction.

Each of INT0 to INT3 can be set up to be either "edge sensitive" or "level sensitive". You can understand the difference between these two if you consider the situation where, at the end of the interrupt handling routine, the voltage on the interrupt pin is still held high by the external device it is connected to and has not gone low since the routine was entered. In this case, if the interrupt has been made edge sensitive then the handling routine will not be re-entered. On the other hand, if it is level sensitive then it will be re-entered immediately. NMIs have to be edge sensitive. (Think about it.)

The 80186 uses a fairly sophisticated mechanism of prioritizing maskable interrupts. Unlike on the 6502, an interrupt does not clear the I flag to prevent all other interrupts. Instead, each interrupt has a priority level. When the interrupt occurs, the processor notes the priority level and it then prevents any other interrupts with the same or lower priority levels from occurring. Interrupts with a higher priority level are permitted, however. (Once again, you are referred to the data sheet for details of the priority levels. Remember that, as always with priority levels, a lower number means a higher priority.) A consequence of this system is that any interrupt handling routine must include a section of code to make the 80186 clear the interrupt priority register (actually known as the "in-service register"). If it does not do this, then the 80186 will not know that the interrupt has completed, and it will not allow the same one to occur again. To clear the in-service register it is necessary to write a value to the "End Of Interrupt" (or EOI) register, which is located at output port FF22h. The simplest way is to write the value 8000h to this, and all interrupts will be re-enabled.

Use of Hardware Interrupts on the Master 512

On the Master 512, INT0 is connected to the Parasite Interrupt ReQuest (PIRQ) line of the Tube ULA. It is used to signal to the 80186 that the host has written a data byte to Register 1 or Register 4 of the Tube. This is described in more detail below. INT1 is connected, very strangely, to one of the processor's chip select lines, PCS0. This is described in a little more detail in the next section. INT2 and INT3 are not connected, and can have no function. As noted above, the NMI line is connected to the output of Timer 1, and the purpose of this is discussed under Interrupt 2 in the "Interrupts" document.

The Chip Select Lines

These are a collection of output signal lines from the 80186. The system can be set up so that a signal is sent along one of them whenever the processor accesses an area of memory or of I/O space within a particular range. These lines are provided principally for the convenience of hardware designers, and they make it easy for external devices and memory chips to be mapped into I/O space or memory space without the need for complex address-decoding circuitry.

Master 512 use of the Chip Select Lines

On the 512, only four of these chip select lines are connected to anything. The Lower Memory Chip Select (LMCS) and Upper Memory Chip Select (UMCS) control the memory. This is described in more detail in the next section. The second Peripheral Chip Select line (PCS1) is connected to the Tube chip, and it is set up to cause the Tube registers to be mapped to I/O ports 80h to 8Eh.

The other connection is rather an oddity. The PCS0 line, the first Peripheral Chip Select, is connected to the 80186's INT1 line. However, this does not seem to be used by the system and the interrupt is masked off. I can only speculate about the reason for this connection. The connection is a direct one, but PCS0 is active low whereas INT1 is active high. This means that if the interrupt is enabled and made edge sensitive, then whenever the processor accesses an I/O port in the range 0 to 7Fh an Interrupt Dh will occur on the following cycle (*ie* immediately after the instruction). This could in theory be used to trap `IN` and `OUT` instructions to ports in this range. The instructions could then be translated into something the 512 could handle. The PC uses ports in this range for the keyboard interface and the sound chip (and a few system controls). It might be possible to set things up using Interrupt Dh so that writes to the sound chip were made to operate the BBC's sound system, or perhaps to make pop-up programs which intercept Interrupt 9 work normally. I can only guess that the intention of the connection was to implement one or other of these, but in the event nothing came of it. An independent programmer might be able to use this method to write a program which would make the 512's sound work like a PC's, though it looks to me like a non-trivial task.

The RAM and ROM Memory

The "standard" Master 512 comes with 16kb of ROM and 512kb of RAM. The ROM contains code for system initialization, for the 80186 monitor, for accessing the host MOS (through Interrupts 40h to 4Ch) and providing handlers for other standard Tube protocols, for data transfers using OSWORD FAh, and for booting DOS-Plus. In normal circumstances the ROM code is not used after DOS-Plus has been initialized.

The only point about the memory I wish to make in this article is the way that the memory chip select lines are used to structure the memory map.

The system is wired up so that the UMCS line separates the ROM from the RAM. When UMCS is active memory accesses use the ROM, and when it is not active they use the RAM. The ROM itself sets up UMCS so that it is active in the top 256kb of memory space. This means that when the processor accesses a memory location in the range 0 to BFFFFh then the RAM is used, and the ROM is used if the location is in the range C0000h to FFFFFh. (These addresses are 20-bit absolute – nothing to do with the internal segment:offset system of the processor unit.) Since the ROM is only 16kb in size, and so has only 14 address lines, this top 256kb of memory will appear to contain several copies of the ROM.

It might be expected that the 512kb of RAM would be wired up simply using 19 of the 80186's 20 address lines but, in fact, things are not done in that way. The 512's RAM is made up of two 256kb banks. Each of these is wired up to the bottom 18 of the address lines (*ie* lines A0 to A17). However, separation between the two banks is performed using LMCS rather than line A18 of the processor's address bus. LMCS is set up to be active in the bottom 256kb of memory. This has an important consequence. If the simple address lines had been used, then an access to a memory location in the range 80000h to BFFFFh would have used the lower memory bank. As it is, though, when a memory location in this range is used the LMCS line is not activated, and so the upper half of the RAM memory is used. The result of all this is that the memory chips are mapped into the 1Mb of memory map in the following way:

Location		***Memory bank***
00000h to 3FFFFh	:	Lower 256kb of RAM
40000h to 7FFFFh	:	Upper 256kb of RAM
80000h to BFFFFh	:	Upper 256kb of RAM
C0000h to FFFFFh	:	ROM

This is particularly important for the video system. DOS expects RAM memory to be contiguous, starting from location 0, so the memory of a 512kb machine has to run from 0 to 7FFFFh. On the other hand the screen adapter memory of a true PC with CGA runs from B8000h to BFFFFh. Quite a large number of PC programs work by directly writing to (or occasionally reading from) the screen adapter memory. The 512's memory map means that when such a program writes to the screen it affects the bytes at the top of the upper RAM bank, which is at the extreme top end of the 512's RAM memory. If things had been done the other way (using address line A18, mapping locations 80000h to BFFFFh into the lower RAM bank), then the screen memory would have fallen bang in the middle of RAM – just the area DOS needs to use for programs.

It is perhaps worth noting that a small amount of re-wiring has to be done to extend the memory to 1Mb. For this LMCS has to be disconnected and the normal top two address lines used. UMCS is still used to activate the ROM. If nothing further were done, then locations C0000h upwards would still appear as ROM and the system would only be able to find 768kb of the 1Mb. However, a very slight modification of the ROM can be made. (Only two bytes need to be changed.) If this is done then UMCS is made to be active only in the top 64kb of memory, and then locations 0 to EFFFFh will be RAM and only F0000h to FFFFFh will be ROM. (Things could, in theory, be done to make only the top 16kb be ROM, but this would take a greater re-write.) This appears to give the machine 960kb of RAM (or 1008kb if the extra re-write were to be done). Close examination of a 1Mb machine will show that this is indeed the case. However, the initialization procedure tests for RAM in blocks of 128kb, and so it will only find 896kb present. In reality this makes little difference since few 512-compatible programs can use the "high memory" area above the screen memory.

The Tube ULA

The Tube chip is, in reality, simply a fast data link between the two processors. It possesses a number of internal registers, which are just small data buffers. Data can be written on one side and read off the other. Naturally, there are also a few "handshaking" control lines enabling the processors to signal to one another when something needs doing.

The Tube works in more or less the same way whatever the parasite processor happens to be. There are just a few areas, though, in which things are done in a slightly non-standard way on the 512. In summary, on the 512 things are connected up so that:

- The Tube registers occupy even-numbered I/O ports from port 80h to port 8Eh. (There is a small error in the circuit diagram provided with Robin Burton's book. The address lines connected to the Tube ULA on the parasite side are actually numbers 1, 2 and 3 not 0, 1 and 2, causing the Tube registers to occupy the even-numbered ports.)
- The Parasite IRQ line from the Tube is connected to INT0 of the 80186. This means that an interrupt signal from the host always causes an INT Ch in the 80186. (The 80186's other interrupt lines, INT1 to INT3 are not connected on the Master 512, so hardware interrupts Dh to Fh do not occur.)
- The Tube Data ReQuest line (DRQ) is connected to to DRQ0 line of the 80186, the control signal line for DMA0. This is different from a "standard" second processor (such as the 6502) in which this line is not connected.
- The Parasite NMI line from the Tube chip is not connected on the Master 512. Again this is different from a standard second processor, in which this line is connected to the processor's NMI input. (The 80186's NMI input in the 512 has a totally different use.)

The Tube Registers

Most descriptions of the Tube state that there are 4 data registers. It really makes more sense to say that there are 8 registers, all quite independent of one another, four conveying data bytes from the host to the parasite, and four which take data the other way. The confusion arises because there are four registers in each direction, and they are mapped to the same addresses. However, data travelling in the two directions will not interact, so really the host-to-parasite and parasite-to-host registers are quite separate.

For each processor, 4 of the registers appear as "read" registers and 4 as "write" registers. (Those that are "read" for one processor are, of course, "write" for the other and *vice versa*.) A byte is entered into a "write" register simply by writing it to the mapped address. Reading the address (provided that there is at least one byte on the "read" register) causes a byte to be removed from that register. Note that nothing further needs to be done to clear the byte from the register. This means that the Tube registers are "read-sensitive".

Each register can hold at least one byte, and some can hold more. Those that can hold more than one act as FIFO buffers.

Each data register has one status bit accessible from each processor. In every case the bit is set if the register can be used. Thus for each "read" register the bit is set if data is available in the register (*ie* a read is possible). Similarly, for each "write" register the bit is set if there is at least one byte of space in the register (*ie* a write is possible, though see the comment on PH3 below). Just like the data registers, the status bits are doubled-up in the available addresses. The status bit for each "read" register is mapped into bit 7 of the corresponding address, and the status bit for each "write" register is mapped into bit 6 of the address.

The above is all valid for both processors.

There are also 6 control bits. These are mapped into bits 0 to 5 at the same address as the status bits for the two No 1 registers in the host. (They are only accessed by the host and the parasite does not read or write them.)

The location in the memory map on the host side of the data registers and the status and control bits is well documented. To summarize, the locations used are &FEE0 to &FEE7. The data registers are at the odd numbered addresses, and the corresponding status bits for each are at the preceding address. The control bits are found in location &FEE0.

On the co-processor side of the 512, 8 locations are also used and in the same order. These are in 80186 I/O-space starting at location 80h, and only even numbered locations are used. (Thus the status bits for the No 1 registers are at port 80h, the No 1 data registers at port 82h, *etc*.)

In what follows, the data registers are referred to as HP1, HP2, *etc* for the host-to-parasite registers, and PH1, *etc* for the parasite-to-host registers.

Register Characteristics

PH1	This acts as a 20-byte FIFO buffer.
HP1	This is a single-byte register. If control bit 1 is set, placing a byte on this register causes an IRQ in the parasite. On the 512 a signal is generated on INT0 so an Interrupt Ch is caused in the 80186.
PH2	Single-byte register.
HP2	Single-byte register.
PH3	This operates (in a rather untidy way) as a 1-byte or 2-byte register. It can always accept two bytes, but the "space available" status bit (on the parasite side) is cleared as soon as one byte is written to the register. In other words, the register is able to accept a second byte even though the status bit suggests that it cannot. When this register becomes empty the Tube DRQ line goes high. On the 512 this sends a signal to DMA0 via the DRQ0 pin of the 80186. Also, if control bit 3 is set and the co-processor NMI is connected, then an NMI is generated at the same time. The NMI is not connected on the 512, however, so this control bit has no effect. Note that control bit 4 does not affect the size of this register nor the operation of the status bits. It does, however, modify the action of the DRQ line. See below for further details.
HP3	If control bit 4 is set, this register acts as a 2-byte buffer. If the bit is clear the register will hold only one byte. When this buffer is full the DRQ line goes high. Again, control bit 3 being set causes a signal from the Tube NMI on the parasite side, but this does nothing on the 512.
PH4	This is a single-byte register. According to other documentation, if control bit 0 is set then an IRQ will be caused in the host when this is written to. I have not tried this out and cannot verify it one way or the other.
HP4	Another single-byte register. If control bit 2 is set writing a byte to this register causes an IRQ in the parasite. Again, on the 512 it will be Interrupt Ch caused by a pulse on INT0.

Action of Bits in the Control Register

Bits 0 to 5 of this register (at location &FEE0 in the host) control various aspects of Tube operation.

The first thing to realise about these bits is that setting or clearing them is not carried out by the simple method of merely writing the desired value to location &FEE0. Rather it is done as follows: To set a bit (or bits) you write to location &FEE0 a byte in which the bit (or bits) to be affected must be set and in which Bit 7 is also set. To clear bits, you write a byte in which the bits to be affected are again set, but in which Bit 7 is clear. If a bit in the byte written is clear, then the corresponding control bit will not be changed by the write. So, for example, to set control bits 1 and 4 (values 2 and &10) you must write the value &92 to location &FEE0. To clear the same two bits and leave all other control bits unaffected, you would write the value &12 to this location. If you are familiar with programming the BBC Micro you may notice that this method is the same as that used to set up the interrupt enable flags in the 6522 VIA.

The meanings of the control bits are not always quite as some of the available documentation states. Their functions are actually as follows:

bit 0 : If set a host IRQ occurs when a byte is written to PH4. [At least this is what the documentation implies will happen. I have not tried it out yet.] The bit is always clear in normal use.

bit 1 : If set a parasite interrupt occurs when the host writes to HP1. On the 512 the Tube interrupt line is connected to the INT0 line of the 80186, and so such a write will cause an INT Ch. This bit is always set in normal use.

bit 2 : If set a parasite interrupt occurs on a write to HP4. Since there is only one parasite IRQ line from the Tube chip, this will also cause interrupt number Ch. This bit also is always set in normal use.

bit 3 : This bit has no effect at all on the Master 512. According to the documentation it should cause a parasite NMI on a host read or write to or from one of the No 3 registers (see above), but these NMIs do not operate on the 512. This bit is set for data transfer types 0-3, clear for types 4-7 (see below).

bit 4 : If this bit is set then HP3 operates as a 2-byte buffer, if clear this register will hold only one byte. Note: This bit does not affect the size of PH3 which always operates as described above. The bit does however alter the way that both these registers affect the DRQ handshaking line. See below for more details. The bit is set for transfer types 2 and 3, cleared for other transfer types.

bit 5 : Parasite reset. Note that the documentation regarding this is misleading. According to the documentation reset is active low. Certainly at the microprocessor itself reset is active low. However it is setting this bit high that brings the processor reset line low, so this bit is active high. To cause a parasite reset write first 1 then 0 to this bit. This is done by writing first &A0 and then &20 to location &FEE0 in the host. The bit is always clear in normal use, and must be clear for the co-processor to operate (otherwise it is held in continuous reset). For an 80186 reset, this bit must be held high for at least 6 co-processor clock cycles, but the 6502 cannot operate faster than this anyway. It may need to be held high for a longer time with other co-processors.

In Summary

On the Master 512, the standard control bit settings along with the way the co-pro is wired up, mean that writes to HP1 and HP4 always cause an Interrupt Ch in the 80186. Reads to or writes from the No 3 registers by the host do not cause NMIs in the parasite (though they do affect DMA0). Writes from parasite to host never cause a host interrupt.

Software Support

If the Tube hardware is to be made to work, it naturally needs some controlling software. Since the Tube itself operates as a link between two separate processors, there has to be controlling software on either side. There is thus some "Tube code" in the memory of the host (6502 code, of course), and some in the parasite memory (and on the 512 it is, naturally, in 80186 code).

In order for this to be used, three sets of software protocols have to exist. These are:

- Protocols to enable other software in the host (MOS, filing systems, *etc*) to communicate with the "Tube host code".
- Protocols to enable other software in the parasite (DOS and user programs) to communicate with the Tube controlling code in the parasite.
- Protocols to allow the two parts of the Tube control code to communicate with one another across the Tube.

On the Master 512 the situation is made rather more complicated since both parts of the Tube control code are made up of different sections, and these must communicate correctly with one another as well as with other software.

The code in the BBC Computer host for the 512 is in three sections:

- First there is the standard "Tube host code" which is used by all varieties of co-processor. This is kept in the host ROM (in the DNFS chip on a Model B, in ROM slot &F of the system ROM on a Master). At reset the host MOS code tests for the presence of the Tube. (It does this by trying to alter some of the control bits found at location &FEE0 (see above), and if it succeeds then it knows that the Tube is present.) If it finds that the Tube is active then it issues Sideways ROM Service Call &FF. The service call is intercepted by the appropriate ROM, which then copies the "Tube host code" into what would otherwise be the language workspace (Memory Pages 4 to 6 and the lower part of Page Zero).
- The second part of the host code on the 512 is the code to handle OSWORD FAh. This is stored in the ROM on the 512 side of the Tube. On reset it is copied across the Tube into the host, and resides in Memory Pages &25 and &26. Warning: It has bugs in it – see the discussion in Appendix F of the *Master 512 Technical Guide*. (It is stored in ROM to make it available for the 80186 monitor's `*TFER` command.)
- The third section is stored on the DOS-Plus boot disk in the file called 6502.SYS. When DOS is booted up, this code is copied across the Tube into the host (having already been copied across the Tube into the parasite from the disk) and is placed in Memory Pages &28 to &2C. It handles a number of host functions needed by the 512 system, and is described in detail in Chapter 5 and Appendix F of the *Master 512 Technical Guide*.

In the parasite things are rather different. There are two parts to the code on that side, but it would be more correct to say that there were two versions rather than two sections. First there is the Tube handling code in the 80186 ROM. This is used by the 80186 monitor and (to a small extent) on system boot. It would also be available for use by any programs written to operate directly from the monitor rather than from DOS (if anybody wanted to write one!). Note that it again contains bugs.

The other part of the parasite Tube handling code is an integral part of DOS-Plus. When DOS-Plus is running, it does not use the 512's ROM code at all. Rather it supplies its own routines for Tube handling. These are similar to the ROM-based ones, but they are not identical. The versions within DOS-Plus appear to be better written than the ROM ones, and certainly they are more free of bugs.

If you look at the code carefully with a disassembler (or rather with two disassemblers) it is evident that the various sections were written by different people. It is clear that in places some of the programmers did not understand exactly what others were doing, and not all fully understood the detailed operation of the Tube itself. The various sections do not mesh together very tidily. On the whole (and apart from the few bugs) the system works, and remarkably well considering its complexity. However, there are a few places where it seems to operate more by chance than by design.

The Software Protocols

It is not the purpose of this article to describe how to use the Tube software. The Tube software can be accessed either by the host processor or by the parasite. Use of it by host programs is described quite adequately in the literature (the most detail is in the *New Advanced User Guide*). Use of it by 512 programs running under DOS is described elsewhere in this collection, particularly in the documents about DOS-Plus Interrupts and the Use of the XIOS.

However there are a few ways in which the literature is slightly misleading, so below is a brief summary of the protocols that exist, together with a few comments about their implementation on the Master 512.

All the host-initiated protocols which actually use the Tube begin with a write to HP1 or HP4. Since control bits 1 and 2 are always set, this causes an interrupt in the parasite (INT Ch on the 512). The interrupt handling routine then takes over in the co-processor, and the protocol is completed within this routine. Note that, by contrast, protocols initiated in the parasite do not cause interrupts in the host.

Host-Initiated Protocols

The Tube protocols which begin in the host are as follows:

Tube Claim and Release

Carried out by the standard Tube Host Code, entry point at &406. (Entry is with a caller ID value plus &C0 (for Tube claim) or plus &80 (for release) in the A register; C is set on exit if the claim is successful.)

A Tube claim is handled entirely by the host. Nothing is passed across the Tube. The parasite is informed of the Tube owner when a genuine data transfer is initiated, though the 512 makes no use of this information. A Tube release uses a version of the "Data Transfer" protocol with a code number of 5 to inform the parasite of what has happened. The 512 takes no action. (Note that a totally different Tube claim and release is required if a DOS-Plus program needs to use the Tube. These calls have a different purpose from and are completely independent of the host-initiated Tube claim and release.)

Data Transfer

This again is initiated by a call to the routine at address &406 (with data transfer type – 0 to 7 – in the A register). Details of how to use it are adequately described in the published literature. A description of how it actually works is given below. The interrupt is created in the parasite by writing the type (which must have the top bit clear) to register HP4.

Eight protocols are provided by the system. Types 0 to 3, 6 and 7 are truly for data transfer. The books explain these. The descriptions in the literature of protocol type 4 are somewhat misleading, and a little more is said about this just below. Type 5 is for system use only, and is used to inform the co-processor of a Tube release (see above).

There are a couple of things that are worth noting about true data transfers on the Master 512. The first is that data transfer types 2 and 3 (double-byte transfers) are unreliable on this machine. For reliable data transfer use types 0, 1, 6 and 7 only. (This applies to parasite-initiated data transfer using OSWORD FAh as well.) The second point is that the 80186 is a much faster and more powerful processor than the 6502. You do not need to worry about the official transfer delay times. The 512 will supply or read data as fast as the 6502 can handle it, so you do not need any NOP instructions in the loops on the host side to slow things down. One result of this second fact is that nothing is lost by the lack of transfer types 2 and 3. With the 512, types 0 and 1 are just as good.

Protocol type 4 is not a true data transfer. According to the books this is supposed to cause the co-processor to jump to the location specified with the protocol. On the Master 512 things do not work in that way. The reaction of the parasite to a call using this protocol type depends on whether it is the ROM or DOS-Plus which is in control:

If DOS is active, then the call does nothing.

If the ROM is in charge (during system boot or when the monitor is running) then things are a bit more complicated. If a call using this protocol is made during the reset sequence then the address given is noted by the system. It is supposed to be the case that in certain circumstances (when DOS is not to be booted up) a jump should be made to this address after the completion of the reset sequence. In practice, whether this happens correctly or not depends on which version of the ROM is present in the 512. (There are different versions of the ROM around – unfortunately they are difficult to tell apart because someone forgot to alter the date in the newer ones. The version number (which can be displayed with `*HELP MON`) is a partial guide, but it looks like Version 1.10 may be earlier than some varieties of Version 1.00!) Some versions use the given address, some just ignore it. If the call is used other than during the reset sequence, then a jump is certainly not made straight away. It may be made at some later time, or there again it may not, depending on the ROM version!

Language Start-up

According to the books, a call to location &400 in the standard Tube host code is used to start a language in the co-processor. This entry point is not intended to be used by user programs. It is actually entered under two circumstances. At the end of the Break sequence control is transferred here with both the C and Z flags clear. This causes the current language code to be copied into the parasite (though only on hard break or power-up), a completion signal is passed to the parasite to start it running, and the host processor enters the "Tube idle loop" on Page Zero.

The other situation which can can cause the code to be entered at location &400 is a language start-up call through OSBYTE with A=&8E. In this case C is set on entry. This operates as the former entry, but copies the language code across unconditionally. For some reason, the code is also entered in this way when the host cannot find a language and "thinks" it is starting up the co-processor in CLI mode.

Language start-up is handled by the software as a series of Type 7 data transfers (bytes of 256 bytes, host to parasite), followed by a Type 4 (jump to given address).

On the Master 512, this language start-up is unreliable. The code is copied to the co-processor correctly, but whether or not it is actually entered depends, as above, on the ROM version in use. In reality, no-one has ever produced a language ROM for use on the 512, so we are not missing much by the failure of this feature to operate correctly.

Escape update

A simple protocol exists for the MOS to update the Escape flag from the host to the parasite. A negative byte (*ie* top bit set) is written to HP1. The Master 512's ROM takes note of this, and the monitor will respond to Escape. DOS-Plus ignores the Escape flag value.

Error reporting

When an error occurs in the host it is reported to the parasite by the BBC MOS. The information transfer begins by writing a negative byte to HP4. Again, the ROM and DOS-Plus handle this differently. The ROM causes an error message to be displayed, and returns control to the monitor. DOS-Plus notes the error, but takes no further action. See the description of XIOS call 87h for more about DOS-Plus' handling of host errors.

Events

If an event has been enabled (by *FX 14 or equivalent) then its occurrence will be reported to the parasite, starting by writing a positive byte (top bit clear) to HP1. When DOS-Plus is running on the 512 then there are three kinds of events that are enabled and which may occur. These are Events numbered 0, 4 and 10.

Event 0 is the "Output buffer empty" event. When this occurs DOS-Plus wakes up the printer driver, and if there are characters waiting to be printed they are written across the Tube to fill up the host's printer buffer again. This means that, from the point of view of DOS-Plus, characters are printed in batches rather than one by one, and thus some time is saved. (Note that when the 512's DOS-Plus wants to print anything it tests the space available in the printer buffer in the standard way using OSBYTE 80h. However, it then discards the high byte of the result, assuming that the printer buffer is always less than 256 bytes. This means that if you have implemented a larger printer buffer in the host (*eg* using Sideways RAM) DOS-Plus will not use it properly unless you "tweak" it slightly to make sure that it never reports the amount of free space as a multiple of 256 bytes.)

Event 4 is the "Vertical Sync" event. This occurs 50 times per second. Master 512 DOS-Plus does almost all of its background processing on this event. This includes checking the keyboard, updating the screen, adjusting the system clock, noting the mouse position (if enabled), moving the cursor, *etc*.

Event 10 is not a standard host event. It is generated by the 6502.SYS part of the host code on completion of a floppy disk action.

Summary of Host-Initiated Interrupts

Each of the above protocols communicates information to the parasite by writing a byte to HP1 or HP4, causing an 80186 Interrupt Ch on the Master 512. In every case the protocol is completed within the parasite interrupt routine. The top bit of the first byte written, and the register used, will distinguish between them. There are thus four principal host-to-parasite protocols, which operate as follows:

Protocol	***Register used***	***Top bit of first byte***
Event	HP1	clear
Escape update	HP1	set
Data transfer	HP4	clear
Error report	HP4	set

Parasite-Initiated Protocols

The parasite does not interrupt the host. On any two-processor BBC Computer system the host spends most of its time in a little loop waiting for bytes to be written to PH1 or PH2. When one of these occurs it will take appropriate action, and at the end of the protocol return to the little loop. The only exception is when an internal host interrupt occurs. Any of these happenings may, of course, launch one or more of the host-to-parasite protocols listed in the previous section.

The parasite-initiated protocols are all calls to the standard BBC MOS routines, and all are adequately documented elsewhere. Calls to OSWRCH are caused by writing a byte to PH1, calls to all the other MOS routines are started by writing a byte to PH2.

The only thing to note about these routines on the Master 512 is that DOS-Plus uses the MOS routines very sparingly, and when it does use them it does not normally use the code in the 80186 ROM. (See Interrupts 40h to 4Ch for more about these calls.)

Most communication from DOS-Plus to the host is done using the 512's dedicated OSWORDs, with action codes &FA to &FF. So each of these protocols begins with writing the value 8 (code for OSWORD) to PH2, followed by the action code written to the same register. Because the action codes are all above &E0 this will cause a jump in the host *via* the vector USERV, and the Master 512's dedicated host code takes over to complete the protocol. In brief, these protocols are:

OSWORD number	***Function***
&FA	Data transfer
&FB	Floppy disk read, write, *etc*
&FC	Write to CRTC registers (for cursor position, scrolling, *etc*)
&FD	*<not used>*
&FE	*<not used>*
&FF	Write block of data to host screen

Of these, as noted above, OSWORD &FA is implemented in (buggy) code downloaded from the 80186 ROM, whereas code

Of these, as noted above, OSWORD &FA is implemented in (buggy) code downloaded from the 80186 ROM, whereas code for the others is contained in the DOS-Plus boot file 6502.SYS.

As you can see, two values within this range are not actually used. It seems that there was a purpose for them in the original conception, but in the end they were not needed. OSWORD &FD was originally intended for text output, but in fact under DOS-Plus everything is converted to graphics before it is written to the screen. OSWORD &FE was to be used for hard disk accesses, but the programmers elected instead to perform all these through calls to ADFS. OSWORD &FA is not actually ever used by DOS-Plus.

More detail about all of these can be found in the *Master 512 Technical Guide*.

Operation of Data Transfer *via* Tube Register 3

Most of the protocols described in the previous section work fairly straightforwardly. An examination of the Tube host code yields few surprises. There is one area, though, where things are obscure, and the details of the Tube control code (on both sides) appear tortuous. This is the handling of data transfers as initiated by a call to location &406 in the host.

For most people the remainder of this document is for interest only. It is only going to be of positive use to people wishing to do "non-standard" things with the Tube. For anyone, though, it may help to clarify some odd points about how the Tube works.

Data transfer is initiated by the host and occurs according one of six protocols. These are types 0 to 3, 6 and 7. The meanings of the six transfer types are well documented. In summary: the even numbered ones (0, 2 and 6) are from parasite to host, the odd numbered ones (1, 3 and 7) are from host to parasite. Types 0 and 1 allow an unspecified number of single bytes, types 2 and 3 an unspecified number of pairs of bytes, types 6 and 7 exactly 256 bytes.

The protocol of the initialization between the software on the two sides of the Tube is that a sequence of bytes is written from the host to the parasite on HP4. (Each time the host must wait until the register is clear by monitoring the status bit.) After this the host reads or writes the data to or from HP3 or PH3 as appropriate. Data transfer has a guaranteed minimum speed, so the time-wasting monitoring of status bits is avoided.

The initialization bytes (in the order they are sent across HP4) are:

- The transfer type
- The Tube owner ID byte. The Tube owner ID is a 6-bit number (as allocated by Acorn). The code in the host will normally have been informed of the ID value by an earlier Tube claim. The byte sent has the ID number in the bottom 6 bits and the top two bits are set. (The 512's software ignores the value of this byte.)
- The data transfer address. 4 bytes, high byte first. With the 512 the first two bytes are taken as the segment value, the last two as the offset value.
- One more byte (value irrelevant) sent as a synchronization signal.

Note that "transfer type 4" (jump to address) uses the same protocol, but no data is sent. For type 5 (Tube release) only two bytes are sent: the type number and the ID byte.

Transfer types 6 and 7 are handled entirely by normal software in the parasite, without using any form of interrupts or other signalling. Types 0 to 3 are different. The "official" way of implementing these is by using NMIs in the co-processor, and the Tube host code is designed assuming that things are done in this way. The 80186 co-processor, however, is put together slightly differently. As noted earlier, the NMI output line from the Tube ULA is not connected (and the NMI input line on the 80186 has a different function). Data transfer is performed using one of the DMAs integral in the 80186 (actually DMA0). This is synchronized by signals on the DRQ0 line, and to provide these the line is connected to the DRQ output line from the Tube ULA. On the whole, the DMA synchronized in this way operates in a very similar way to an NMI. Yet there are subtle differences which lead to complications in the Tube handling code on the parasite side.

Hardware details

The operation of the DRQ line from the Tube is affected somewhat by the state of bit 4 of the Tube control register. The details are first described for when this control bit is clear, and then the differences for when the bit is set are noted.

Note that this line differs from the NMI line in the following two ways:

- It is not affected by bit 3 of the Tube control register. On other co-processors (which use genuine NMIs) it is possible to enable or disable the interrupt by setting or clearing control bit 3. By contrast, there is no way of rendering the DRQ signal inactive.
- All NMIs are edge-triggered. The DRQ is a level-trigger.

Provided that control bit 4 is clear, DRQ operates as follows.

> The DRQ line from the Tube ULA is set high whenever either:
>
> > HP3 contains a byte *or*
> > PH3 is empty.

The basic principle of the DMA data transfer is like this: The DMA is set either to read from input port 8Ah (Register 3) and write to co-processor memory (for types 1 and 3) or to read from memory and write to output port 8Ah (for types 0 and 2). In either case the DRQ control line acts as the (external) synchronization signal.

If a type 0 transfer is underway, the next byte to be passed across from the co-processor to the host is placed on the Tube (PH3) until the host is ready to read it. When the host is ready it reads the byte off. The register is then empty and the DRQ line goes high. This prompts the DMA to put the next byte onto the Tube.

Type 1 transfers work in the converse way. A byte is written by the host onto HP3. This causes the DRQ line to go high, and the byte is then read off by the DMA.

Because the control line operates in this manner, during parasite-to-host transfers the register HP3 (*ie* the No. 3 register in the opposite direction which is not in use for the transfer) must be empty. If it were not then the DRQ line would be held high continuously and so the DMA would run free and uncontrolled. On the other hand, though for the same reason, during host-to-parasite transfers the register PH3 must contain a byte. This means that for all parasite-to-host transfers an extra byte must be written after the last byte required. This byte (it may sometimes be two bytes – see below) must be left on the register so that subsequent host-to-parasite transfers will work, but it must be pulled off PH3 and discarded at the start of the next parasite-to-host transfer. This means that on type 6 transfers the parasite must write 257 bytes to the register, the 256 bytes to be transferred and one extra one to keep PH3 filled.

There is a further slight complication with parasite-to-host transfers. When a byte is written by the DMA to the Tube, the DRQ line goes low. However this does not always take place quite immediately. Sometimes (more often than not) it remains high long enough to prompt a second DMA transfer. There are then two bytes on the PH3 register. This means that at any time there may be either one byte or two bytes on this register waiting to be read by the host. A future parasite-to-host transfer may thus have to remove either one or two bytes from PH3 before reading the bytes it has requested.

This describes the way things work for data transfer types 0 and 1, when control bit 4 is clear. If control bit 4 is set, as it is during type 2 and 3 transfers, then things are a little more complex. In the host-to-parasite direction the line goes high when two bytes have been written to HP3. It does not go low again until both have been pulled off. The equivalent happens in the opposite direction. The line goes high when PH3 becomes empty, but does not go low again until two bytes have been written to it. There seems to be some unreliability about this. (I suspect that as with type 0 and 1 transfers it is probably that the DRQ line remains high a little too long, sometimes prompting the write of a third byte which is then lost.) The result is that type 2 and 3 transfers do not work perfectly on the Master 512.

Handling of Data Transfer Protocols by Parasite Software

Data transfer is initiated by a write (of the type code) to HP4. This initiates an INT Ch sequence in the 512.

Data Transfers using the 80186 ROM

When the ROM is in control the 80186 then behaves as follows:

- Clear the interrupt in-service flag (by writing 8000h to the 80186 End-of-Interrupt (EOI) register)
- Check that the interrupt was caused by a write to Register 4 (rather than Register 1) and that bit 7 of the byte written is clear (so the protocol is for data transfer rather than error reporting)
- Read one byte (transfer type) from HP4 and store it
- Switch off the DMA (by writing a code value to the DMA0 control register)
- Read one byte (Tube claim identifier) from HP4 and discard value
- ***If*** type = 5: exit.

  Otherwise: continue
- Read next 4 bytes (transfer address) from HP4 and store
- ***If*** type = 0 or 2:

  - Write address (converted to 20-bit absolute) to DMA source pointer
  - Write value 8Ah (output port – for Register PH3) to DMA destination pointer
  - Start DMA by writing appropriate code value to DMA control register

  If type = 1 or 3:

  - Write address (converted to 20-bit absolute) to DMA destination pointer
  - Write value 8Ah (input port – for Register HP3) to DMA source pointer
  - Start DMA by writing appropriate code value to DMA control register
- Read one byte (sync) from HP4 and discard value
- ***If*** type = 6:

  - Send 101h bytes from the specified address onwards to PH3

  If type = 7:

  - Read 100h bytes from HP3 and store at the specified address onwards

You may like to note a few things about this sequence:

- There is no difference (apart from internally in the Tube chip) between transfer types 0 and 2, nor between types 1 and 3.
- The DMA starts before the sync byte is read. Type 6 and 7 transfers do not start until after this.
- Type 6 transfers send 101h bytes so that, for the reason mentioned above, a byte is left on the PH3 register after the transfer. DMA-driven parasite to host transfers automatically keep this register filled.

Data Transfers under DOS-Plus

The above describes the action when the machine is under the control of the co-processor ROM. When DOS-Plus is in control it replaces the routines with some of its own. These differ from the ROM-based routines in the following two ways:

- For transfer types 0 to 3 the DMA is not started until ***after*** the sync byte has been read.
- During transfer types 6 and 7 the internal timer (Timer 1) connected to the 80186 NMI input line is disabled. This is clearly necessary, since an NMI occurring in the middle of a data transfer would upset the timing. The timer does not need to be disabled during transfers of types 0 to 3 since the DMA activity takes precedence over the CPU.

Handling of Data Transfer Protocols by Tube Host Code

The initialization of Tube data transfers is performed by entry to the Tube code at location &406, with the transfer type (0 to 7 but not 5) in the A register.

The code was written to work with any co-processor, and not specifically for the Master 512. It works with the 512, but possibly not always for the reasons intended.

The code uses one or two little tricks, especially with the numbers loaded in from the translation table at location &518 – the same location in both the Model B and Master hosts. (Note that something has gone wrong at these points with the disassembly listings in Appendix E of the *Master 512 Technical Guide* – these bytes are not all zero, and neither are they temporary data!) The techniques used make the code rather difficult to decipher. The explanation is this:

The different transfer types have the following characteristics:

- Transfer types 0, 2 and 6 are parasite-to-host transfers, so require the byte(s) already on PH3 to be cleared off as part of the initialization process, as described above.
- Types 1 and 3 are host-to-parasite transfers using the DMA (or NMI on other co-processors) and so for the same reason they must leave a byte on PH3.
- Transfer types 4 and 7 are themselves unaffected by whether or not there is a byte on PH3. However these two types must not pull off the byte(s) there as this would disable subsequent use of transfer types 1 and 3.
- Types 0 to 3 are designed to run using NMIs, so bit 3 of the Control Register must be set. Types 6 and 7 do not use NMIs, so this bit must be clear. Type 4 does not cause a transfer, so the value of this bit is irrelevant, but in fact it is cleared. In reality on the Master 512 this bit makes no difference but, of course, the Tube host code is designed for all processors.
- Types 2 and 3 use "2-byte mode", and so require bit 4 of the Control Register to be set. All other transfer types require this bit to be clear.

Read the next section alongside a disassembly listing of the code. The action addresses are given in [brackets]. The address values are given for the Master version. The Model B and B+ version of the code is very slightly different.

The code is entered at location &406. The code for data transfer initialization begins at &433. Significant actions in the process are as follows:

- Disable interrupts [&434]
- Write to HP4 (when there is room on the register):
  - The transfer type number [&439]
  - The Tube-holder ID byte [&43F]
  - The address for transfer (4 bytes) [&442]
- Clear control bits 3 and 4, by writing &18 to &FEE0 [&44A]
- Set the appropriate bits in the Control Register (bit 3 for transfer types 1 and 3, bit 4 for types 2 and 3) [&44F to &452]
- ***If*** transfer type = 0, 2 or 6 (parasite-to-host transfer) pull the extra bytes off PH3. (This is done with two BIT instructions so that the 6502 registers are not affected.) [&455 to &45C]
- Send a byte across HP4 as sync byte and wait until it has been read off by the parasite before continuing [&45F to &465]
- ***If*** action was "transfer type 4", release Tube and restart by direct jump into idle loop [&46D to &473]
- Set bit 3 of the control register for those transfer types which need it but for which it has not already been set. (These are types 0 and 2.) [&476 to &47B]

The following points are perhaps worth noting about this sequence:

- The detail of some of the actions above is controlled by a type-dependent byte read from a translation table at location &518. The values read are:

Type	***Value***
0	&86
1	&88
2	&96
3	&98
4	&18
5	&18 (unused)
6	&82
7	&18

  The meanings of the significant bits in this code byte are as follows:

Bit	***Meaning***
1 (value 2)	Action is a parasite-to-host transfer, so byte(s) must be pulled off PH3 before data transfer commences. (In fact two bytes are always pulled off, since if there was only one byte on the register no harm is done by a second read.)
2 (value 4)	This is only set if bit 1 is also set. If it is set it means that control bit 3 is to be set after the byte(s) have been pulled off PH3.
3 (value 8)	If this is set and bit 7 is also set then control bit 3 is to be set before the sync byte is written. If it is clear, or if it is set but bit 7 is clear, then control bit 3 is not to be set at this stage.
4 (value &10)	If this is set and bit 7 is also set, then control bit 4 is to be set. If it is clear, or if it is set but bit 7 is clear, then control bit 4 is to be clear.
7 (value &80)	This bit must be set if bit 1 is set. It also controls the interpretation of bits 3 and 4.

  The reason for this complexity is that to save space the programmer elected to use a single byte for several controls. The byte is written to location &FEE0 to set whichever of control bits 3 and 4 are required to be set prior to sending the sync byte. (Both of these control bits have already been cleared.) However, bits 1 and 2 of this code byte have nothing to do with control bits 1 and 2. Control bits 1 and 2 must always be set, so it is important that if either of these bits needs to be set in the code byte then bit 7 must also be set so that writing the byte to location &FEE0 leaves control bits 1 and 2 unaffected.

- The above code should not be time-dependent. However it ***is*** time-dependent when used with the Master 512, though only for transfer types 0 and 2 and only when the 80186 is using the ROM-based transfer routines. The reason for this is as follows:

  > For parasite to host transfers PH3 must be cleared before the genuine transfers begin. This is always done by executing two reads (actually BIT instructions) of location &FEE5. There may, though, only be one byte on PH3. No harm is done by the extra read provided that the DMA has not already started. With co-processors using genuine NMIs, control bit 3 is clear at this point and so the NMI is disabled. Thus the second read can never have any effect. With the 80186 co-processor things are otherwise. As noted a number of times already, the Master 512 co-processor uses the DRQ line rather than the NMI, and this cannot be disabled.
  >
  > The 80186 ROM code processes the transfer address and then starts up the DMA as soon as it has received the fourth address byte. The 6502 code sends the fourth address byte before it clears PH3. It is thus vital that the two bytes are pulled off before the 80186 has had time to process the address and start up the DMA.
  >
  > If the 6502 code were too slow and PH3 had only contained one byte, then by the time the second read took place the DMA would have put the first byte of the transfer onto the register. This would then be read and discarded by the 6502. This would result in the byte being lost. The first byte passed to the software would then be the second byte that should have been, and so on.

  The above does not create any difficulties in normal use. Interrupts are disabled during the initialization routine, and the clearing reads take place almost immediately after the address has been written.

  The one condition where this could conceivably cause a problem would be if the routine needed to be used in a situation where NMIs might happen at any time. If this were so, then an NMI could occur at some point between writing the fourth address byte to HP4 and reading the second rubbish byte from PH3. If this happened it would slow up the 6502's action sufficiently that the problem mentioned above would occur. The only solution to this if the 512 needed to be used under such circumstances would be to re-write either the 6502 code or the 80186 code (or both) so that things were done in a different order.

  This problem cannot occur when DOS-Plus is in charge. In the DOS-Plus version of the routines the DMA is not started until after the sync byte has been received. This guarantees that the rubbish will have been removed before transfers start.

Compare the two sequences of actions in the Master 512 (that in the host and that in the parasite). If you do this you will perhaps see why I think that in some ways the system works more by luck than judgement.